Thermal Imaging and CNN-based Machine Learning for Knee Pain Assessment:

By Shiri Guniman and Nir Lapidot

Tabular Data Preprocessing and Initial Analysis:

In the first stage of the project, preliminary preprocessing was performed on the clinical tabular data in order to create a clean, consistent, and structured dataset that could later be integrated with the thermal image data. The data was loaded from an Excel file containing patient identifiers, pain labels, and clinical information. The Excel file consisted of two sheets: Pain Map Labels and VAS.

First, the VAS table was prepared for processing and merging with the remaining clinical data. Since the original Excel file contained column names that were divided according to visit stages and measurement types, the column names were standardized so that each column received a complete, clear, and consistent name. The main columns were then normalized, particularly the visit column, which was marked as Visit, and the patient identifier column, which was marked as PN. As part of this normalization process, unnecessary spaces were removed, values were converted to uppercase, and inconsistent identifier formats were corrected, for example by converting identifiers such as GB-1 into a uniform format such as GB-01. This step was necessary to enable accurate merging between the VAS table and the label table based on patient identifier and visit.

The VAS table was then reshaped from a wide format into a long format. Instead of having the pain measurements for each visit appear in separate columns, a new structure was created in which each row represented a specific combination of patient and visit. The visit stages were defined according to the following mapping: Visit 1 — Baseline, Visit 2 — End of Treatment, and Visit 3 — End of Trial. This structure enabled a clear and systematic connection between the VAS data and the label table.

Next, the label table was merged with the VAS table using two fields: PN and Visit. This allowed the relevant pain measurements to be attached to each clinical observation according to the specific patient and visit.

Following the merge, the pain measurements were adjusted according to the relevant knee side. The VAS table included separate measurements for the right and left knees, such as R-Worst, L-Worst, R-Average, and L-Average. Based on the side column of each observation, two unified columns were created: Worst and Average. As a result, each row contained the pain measurements corresponding to the specific knee to which the observation referred.

Categorical variables were then encoded into numerical variables. The knee side was encoded into a column named side_right, where the right knee was coded as 1 and the left knee as 0. In addition, sex was encoded into a column named male, where male was coded as 1 and female as 0. This encoding was required to enable convenient use of the data during the analysis and modeling stages.

The target variable of the project was created from the AKP column and stored in a new column named pain_label. This variable served as the main classification label, based on which the model would later be trained. After its creation, the distribution of its values was examined in order to understand the number of observations in each class and the degree of balance between the classes.

Subsequently, observations that were not suitable for further analysis were filtered out. Rows in which PF_DIAGNOSIS == 0 or QUALITY == 0 were removed, meaning observations in which the diagnosis was not valid or the data quality was insufficient. The purpose of this filtering stage was to ensure that the rest of the process would be based only on reliable and valid data.

At the end of the process, the processed table was saved as cleaned_tabular_checkpoint.pkl. This file served as the basis for the next stages of the project, including linking the tabular data with the thermal image data, assigning labels to images, and constructing the full dataset for training, validation, and testing.

Image Preprocessing, Feature Extraction, and Pain Label Assignment:

In the next stage of the project, preprocessing was performed on the thermal image data in order to create an organized file in which each image was represented together with the relevant patient details, visit number, imaging region, knee side, and corresponding pain label. This stage relied on the previously processed tabular dataset, which was used to connect the images with the clinical information.

First, the intermediate tabular file cleaned_tabular_checkpoint.pkl was loaded. This file included the clinical data after the cleaning and preprocessing stages. After loading the file, a check was performed to ensure that the main columns required for the next steps existed in the table: PN, Visit, side_right, and pain_label. In addition, the visit and knee-side columns were converted into numerical values to ensure that the merge with the image data could be performed correctly.

At this stage, 432 clinical records were loaded, representing 76 unique patients. The visit values found in the data were 1, 2, and 3; the knee-side values were 0 and 1; and the pain-label values were 0 and 1.

The thermal image folder, Images, was then scanned, and an initial source table was created for all image files that were found. The scan included common image formats, including jpg, jpeg, png, bmp, tiff, and tif. For each file, the file path, file name, patient folder, imaging region, knee side, image width, and image height were stored.

The imaging region was identified based on the path structure and folder names. Regions labeled as anterior or ant were classified as ANT; regions labeled as posterior or post were classified as POST; regions labeled as lateral, lat, or let were classified as LAT; and regions labeled as medial or med were classified as MED. In addition, when side indicators such as L or R appeared in the folders, they were stored as the relevant knee side.

During the initial scan, 1,607 images belonging to 72 patients were found. The distribution of the images by region was as follows: 490 anterior images of type ANT, 289 posterior images of type POST, 339 medial images of type MED, 259 lateral images of type LAT, and 230 files for which the imaging region could not be identified at that stage.

After the initial scan, a patient identifier validation step was performed in order to compare the patient identifiers found in the image folders with the patient identifiers available in the clinical dataset. The identifiers were standardized into a uniform format, so that identifiers appearing in a number-letter format were converted into the standard letter-number format. For example, an identifier such as 75-YO was standardized to YO-75. Spaces and irrelevant characters were also removed, and all letters were converted to uppercase.

This validation step identified 65 patients who appeared in both the image dataset and the clinical dataset. Seven patient groups appeared in the image folders but did not have a corresponding clinical record. These orphan image groups were therefore removed before the continuation of the process. After filtering these unmatched patient groups, 1,452 images remained for further processing.

After this filtering stage, the distribution of the remaining raw images by region was as follows: 443 ANT images, 267 POST images, 304 MED images, 230 LAT images, and 208 images whose region was still classified as UNKNOWN at that stage.

Next, the visit number was extracted from the file name or path using keywords. Expressions such as baseline, base, visit0, or v0 were classified as Visit 1. Expressions such as fu1, visit1, or v1 were classified as Visit 2. Expressions such as fu3, visit3, or v3 were classified as Visit 3. After this extraction process, 1,395 images could be assigned a valid visit number, while 57 images could not be mapped to a visit and were therefore excluded from further processing.

After preparing the source table, preprocessing was performed on the images themselves. Each image was loaded and then underwent several cleaning and processing operations. First, the logo in the upper-left corner of the image was removed by blacking out a fixed area of the image. Then, hair artifacts were cleaned using a method based on morphological operations and the detection of dark, thin regions, followed by filling the cleaned area using inpainting. Next, the object was separated from the background using a brightness threshold, so that the dark background was removed and mainly the relevant body region was retained.

As part of the preprocessing stage, additional quality and artifact-handling fixes were applied. The largest contour was extracted in order to focus on the relevant knee region and reduce the effect of unrelated artifacts such as hands or reflections. In addition, quality checks were applied to avoid saving invalid or mostly black crops. These checks included a minimum visible-pixel ratio and minimum spatial constraints, in order to ensure that the saved image contained a meaningful anatomical region.

Image enhancement was also performed during the preprocessing stage using CLAHE, which stands for Contrast Limited Adaptive Histogram Equalization. The purpose of this operation was to enhance the local contrast of the thermal images, with the aim of improving the visibility of subtle temperature differences and localized thermal patterns within the knee region. Unlike global contrast enhancement applied to the entire image, CLAHE operates locally on small regions of the image. This approach was considered appropriate for the thermal image data, since potentially relevant temperature differences may be subtle and localized within specific knee regions. In addition, the method limits the intensity of the enhancement in order to avoid excessive amplification of image noise. In the updated preprocessing pipeline, CLAHE was applied as part of the image processing stage, before the final train-validation-test split.

At this stage, special handling was also applied to anterior and posterior images of type ANT and POST, which contain images of two knees rather than one. In the updated pipeline, all ANT and POST images were handled using a dedicated midline-splitting procedure. The splitting point was determined using a multi-stage fallback approach that searched for an appropriate vertical separation point between the two knees. The method first attempted to identify the anatomical gap between the two body regions, and if this was not sufficient, additional fallback rules were used, including a central-region split and, where necessary, an exact-middle split. This was done in order to obtain two separate knee images from each bilateral image whenever possible.

After splitting, each half was cropped separately around the object, checked to ensure that it was not mostly black, enhanced using CLAHE, converted into the appropriate RGB color format, and resized to a uniform size of 256 × 256 pixels. The processed images were saved in the folder Data/Images_Processed_Final with normalized file names that included the patient identifier, visit number, imaging region, capture number, and image side.

The split logic for anterior and posterior images also included a rule for determining the knee side. In anterior images of type ANT, the left half of the image represents the patient's right leg, while the right half represents the patient's left leg. In contrast, in posterior images of type POST, the left half represents the patient's left leg, while the right half represents the patient's right leg. Accordingly, the variables leg_side and side_right were created.

For lateral and medial images of type LAT and MED, no splitting was performed, since these images already referred to a specific side. Therefore, these images underwent cleaning, cropping, CLAHE enhancement, resizing, and saving under a normalized file name, using the side indication that appeared in the original path.

At the end of the preprocessing stage, 1,800 processed images were obtained from 1,395 valid source images. The number of processed images was higher than the number of source images because anterior and posterior images were split into two separate images, one for each knee. During processing, 58 errors or skipped cases were recorded, for example in cases where an image could not be loaded, the split could not be performed correctly, or the resulting image was too dark after cropping.

After the processed images were created, pain labels were assigned to each image. For this purpose, the image table was prepared for merging using the side_right column. The images included 979 images of the left knee and 821 images of the right knee.

At the same time, the clinical table was prepared for merging. From the clinical table, the columns PN, Visit, side_right, and pain_label were retained. The patient identifiers were standardized again to match the same format used in the image table, and the Visit and side_right columns were converted into integers. After removing duplicates, 432 clinical records were available for merging.

The merge between the images and the clinical data was performed using three keys together: PN_norm, Visit, and side_right. In other words, an image received a pain label only if there was a complete match between the patient identifier, visit number, and knee side. The use of all three keys was intended to ensure that the clinical label was assigned to the correct knee of the correct patient at the appropriate visit, rather than merely to the patient in general.

After the merge, strict null filtering was applied. Images that did not have a direct clinical match were removed from further processing. This approach was chosen in order to ensure that the dataset was based only on images with an explicit clinical match.

Following this filtering step, 1,622 labeled images remained. The final distribution consisted of 315 images labeled as no pain, marked as 0, and 1,307 images labeled as pain, marked as 1. This reflects a substantial class imbalance in favor of the pain class.

In the final stage, the image manifest file was created. This file includes all processed images and the accompanying information required for the continuation of the project: patient identifier, standardized identifier, file name, file path, imaging region, knee side, visit number, capture number, pain label, image width, and image height.

The final file was saved as:

labeled_image_manifest.csv

In total, the file includes 1,622 processed and labeled images, of which 768 are ANT images, 384 are POST images, 266 are MED images, and 204 are LAT images. The file includes 64 unique patients, and the class distribution is 315 images in class 0 and 1,307 images in class 1. This indicates a clear bias toward the pain class, since the number of images labeled as pain is substantially higher than the number of images labeled as no pain.

After creating the labeled image dataset, additional quality checks and EDA were performed on the image dataset. The purpose of this stage was to ensure that the data structure was valid, that the images were readable, that the anatomical regions were represented in the dataset, and that there was sufficient alignment between the clinical data and the image files.

First, the distribution of images across the different imaging regions — anterior, posterior, medial, and lateral — was examined. This check was intended to ensure that no imaging region was completely missing or represented by an unusually low or high number of images compared with the other regions. In addition, the distribution of image sizes was examined, including width, height, and aspect ratio. This check was important because deep learning models generally require inputs of a fixed size, and therefore the images were adjusted to a uniform size before the training stage.

The distribution of pain labels in the dataset was then examined. This check was intended to determine whether there was a class imbalance problem. It was found that the pain class was represented by a much larger number of images than the no-pain class. Specifically, class 0 included 315 images, while class 1 included 1,307 images. Therefore, this issue had to be taken into account during model training and evaluation. Such an imbalance may increase the risk that the model will favor the dominant class, and therefore the model should later be evaluated using metrics that do not rely solely on accuracy, but also on measures such as sensitivity, specificity, and ROC-AUC.

After the final dataset was created, it was divided into a training set, validation set, and test set. The split was designed to approximate a 70%, 10%, and 20% division, respectively. In practice, because the split was performed at the patient level rather than at the individual image level, the final proportions were not exactly identical to the target percentages. The final split included 1,154 original images in the training set, 210 original images in the validation set, and 258 original images in the test set.

The split was performed at the patient level rather than at the individual image level. This means that all images belonging to the same patient were assigned to the same set only. This decision was made in order to prevent data leakage between the sets. If images of the same patient appeared in both the training set and the validation or test set, the model could learn patient-specific features rather than general patterns associated with knee pain. Therefore, splitting the data by patient provides a more reliable evaluation of the model’s ability to generalize to new patients.

The patient-level split resulted in 44 patients in the training set, 7 patients in the validation set, and 13 patients in the test set. A leakage verification step confirmed that there was no overlap between the patients in the training, validation, and test sets.

Before augmentation, the training set contained 254 images from class 0 and 900 images from class 1. The validation set contained 47 images from class 0 and 163 images from class 1. The

test set contained 14 images from class 0 and 244 images from class 1. These distributions confirmed that the class imbalance remained present after the patient-level split.

After the split, augmentation was applied to the training set only. The purpose of the augmentation was to increase the number of examples from the minority class and reduce the imbalance in the training data. Importantly, augmentation was not applied to the validation set or to the test set, in order to preserve a clean and unbiased evaluation of the model's performance on real, non-augmented data.

In the updated pipeline, augmentation was applied only to class 0, which represented the minority class in the training set. Since the original training set contained 254 images in class 0 and 900 images in class 1, 646 augmented images were generated from class 0 images. As a result, the final augmented training set became balanced, with 900 images in class 0 and 900 images in class 1, for a total of 1,800 training images.

The augmented images were saved in a separate folder named Data/Images_Augmented_Train. Each augmented image was marked using an is_augmented flag, so that augmented samples could be distinguished from original images. This separation was important for traceability and for ensuring that synthetic data was used only during training.

At the end of the process, separate CSV files were exported for the three sets:

train_split.csv, val_split.csv, test_split.csv

The training CSV includes both the original training images and the augmented class-0 images. The validation and test CSV files include only original, non-augmented images. These files include the image paths, patient details, visit number, imaging region, knee side, pain label, and an is_augmented indication. This completed the data preparation stage for modeling: from raw clinical data and raw thermal images to structured, labeled datasets divided into training, validation, and test sets, with a balanced augmented training set and leakage-free validation and test sets.

Classification Model Construction and CNN Training:

After completing the data preparation and EDA stages, a deep learning model was built for binary classification of knee pain based on thermal images. The objective of the model was to receive a processed thermal image of a knee and predict whether the image belonged to the pain class or the no-pain class, according to the value of the pain_label variable.

For this purpose, a custom Dataset class was created. This class reads the CSV files corresponding to the dataset splits, locates the image paths, loads each image from the disk, converts it into RGB format, applies the required transformations, and assigns it the appropriate pain label. In the training set, both the original images and the augmented images were used, whereas in the validation and test sets only original, non-augmented images were used. This was done in order to preserve a clean and unbiased evaluation of the model's performance on data that was not synthetically modified.

Before being passed into the model, all images were resized to a uniform size of 256 × 256 pixels, converted into tensors, and normalized using a mean and standard deviation of 0.5 for each RGB channel. This ensured that all input images had the same spatial dimensions and numerical scale, which is required for stable and consistent CNN training.

The model used in this stage was a configurable convolutional neural network. The architecture consisted of four convolutional feature-extraction blocks, followed by a fully connected classification head. The convolutional part of the network included convolutional layers, batch normalization, activation functions, max-pooling operations, and spatial dropout. The number of channels increased gradually across the convolutional blocks, allowing the model to learn increasingly complex visual patterns from the thermal images. The final convolutional features were reduced using adaptive average pooling, and then passed into fully connected layers for the final binary classification.

The classification head included a flattening layer, dropout layers, a hidden fully connected layer, an activation function, and a final output layer. Since the task was binary classification, the final layer produced two output scores, corresponding to the two possible classes: class 0 for no pain and class 1 for pain.

Hyperparameter Search and Model Selection:

Before training the final model, a hyperparameter search process was performed in order to identify a suitable model configuration. The purpose of this stage was to compare different combinations of model and training parameters and select the configuration that performed best on the validation set.

The search process was implemented using a configurable search framework. The default method used was random search, although the code also supported full grid search and Optuna-based optimization. Random search was chosen as an efficient compromise between exhaustive exploration and computational feasibility, since checking every possible combination in the full grid would have required substantially more training runs.

The hyperparameters examined during the search included the spatial dropout rate, the classifier dropout rate, the degree of label smoothing, the activation function, the weight decay value, the learning rate, and the decision threshold used for converting predicted probabilities into class labels. The activation function was selected between ReLU and GELU, while the threshold values were examined in order to account for the fact that a fixed threshold of 0.5 is not always optimal, especially in imbalanced binary classification problems.

Each hyperparameter configuration was trained for a maximum of 75 epochs. Early stopping was used during this process, with a patience value of 10 epochs. This means that if the validation performance did not improve for several consecutive epochs, the training of that specific configuration was stopped early. This approach reduced unnecessary computation and helped prevent overfitting during the model-selection stage.

The main criterion used for selecting the best hyperparameter configuration was validation balanced accuracy. This metric was preferred over regular accuracy because the dataset was imbalanced, with the pain class being substantially more frequent than the no-pain class. In such a setting, a model can obtain a high regular accuracy simply by favoring the majority class. Balanced accuracy is more informative because it accounts for performance on both classes and therefore gives a more reliable indication of whether the model is learning to distinguish between pain and no-pain cases.

In addition to balanced accuracy, other validation metrics were calculated, including accuracy, F1-score, precision, recall, and the confusion matrix. These metrics were recorded for each hyperparameter run and saved for later comparison. The best-performing

configuration was saved as a model checkpoint, together with its hyperparameters and validation metrics. This checkpoint was then used as the starting point for the final training stage.

Loss Function, Class Imbalance, and Optimization:

The training process used a cross-entropy-based loss function adapted to support label smoothing and class weighting. Label smoothing was included as a regularization technique. Instead of assigning full confidence to the true class during training, label smoothing slightly softens the target labels. This can reduce overconfidence and improve generalization, particularly when the dataset is limited in size.

Class weights were also incorporated into the loss function. These weights were calculated according to the class distribution in the training set. The purpose of this weighting was to reduce the effect of class imbalance by assigning greater importance to the minority class during optimization. Although the training set had already been balanced through targeted augmentation of class 0, the use of class weights provided an additional safeguard against biased learning.

The optimizer used for training was Adam. The learning rate and weight decay values were taken from the best hyperparameter configuration found during the search stage. Weight decay was used as an additional regularization mechanism, helping to reduce overfitting by penalizing overly large model weights.

Final CNN Training:

After completing the hyperparameter search, the best-performing configuration was selected as the basis for the final CNN model. This configuration corresponded to run 41 and included ReLU activation, a learning rate of 0.001, weight decay of 0.0005, label smoothing of 0.05, spatial dropout of 0.30, classifier dropout of 0.35, and a classification threshold of 0.5. The best checkpoint from this stage, obtained at epoch 15, was used as the starting point for final training, and the final CNN architecture was reconstructed accordingly.

The final training stage was performed for up to 1,000 additional epochs. In each epoch, the model was trained on the training set and evaluated on the validation set. Training loss, training accuracy, validation loss, validation accuracy, F1 scores, and combined loss were calculated. The combined loss, defined as the sum of training loss and validation loss, was used as the final model-selection criterion.

The best model according to this criterion was obtained at epoch 603. At this point, the model achieved a training loss of 0.5118, training accuracy of 0.7856, training F1 score of 0.8035, validation loss of 0.5755, validation accuracy of 0.7762, and validation F1 score of 0.8740. The combined loss was 1.0873, the lowest value observed during training. Therefore, the weights from this epoch were saved as final_finetuned_model.pt and used as the final trained CNN model for test-set prediction and subsequent performance analysis.

Test Prediction Export:

After the final model was selected and saved, it was used to generate predictions for the test set. The test set was kept separate throughout the entire process and was not used during

training, hyperparameter selection, augmentation, or validation-based model selection. This ensured that the test set could serve as an independent evaluation set.

For each test image, the model produced a probability score for class 1, representing the probability that the image belonged to the pain class. The selected decision threshold from the hyperparameter search stage was then applied to convert the predicted probability into a final class prediction. If the probability for class 1 was greater than or equal to the selected threshold, the image was classified as pain; otherwise, it was classified as no pain.

The test predictions were exported into a CSV file named test_predictions.csv. This file included the image path, the true label, the predicted label, and the predicted probability for class 1. Exporting the predictions in this format allowed the model results to be analyzed separately in a Jupyter notebook, including both quantitative evaluation and error analysis.

Results Analysis:

The final results were analyzed using a dedicated results-analysis notebook, which loaded the predictions generated by the trained model and evaluated the model's performance on the held-out test set. The analysis included a classification report, a confusion matrix, an ROC curve with AUC score, and an error analysis of false-positive and false-negative cases. These evaluation tools were used because the test set was highly imbalanced, and therefore overall accuracy alone could provide a misleading interpretation of the model's performance.

The classification report was used to examine the model's performance across the two classes: pain and no pain. This allowed the evaluation to consider not only the overall accuracy, but also how well the model identified each class separately. The confusion matrix was then used to provide a clearer breakdown of the model's predictions, including correct classifications and classification errors.

In addition, the ROC curve and AUC score were used to assess the model's ability to separate between the two classes based on its predicted probabilities. Finally, an error analysis was performed in order to examine the cases in which the model made incorrect predictions, with a particular focus on false-positive and false-negative classifications. Overall, this analysis provided a more complete understanding of the model's behavior on the test set, beyond a single accuracy value.

Final Model Performance:

The final selected model was chosen according to the best checkpoint obtained during the final training stage. During training, the model was evaluated using both training-set and validation-set metrics, including loss, accuracy, F1 score, and combined loss. The combined loss, calculated from the training and validation losses, was used as the final model-selection criterion in order to select a checkpoint that balanced learning from the training data with performance on unseen validation data.

The validation results indicated that the CNN was able to learn relevant visual patterns from the processed thermal knee images. However, the test-set analysis showed that the model's performance was affected by the class imbalance in the data. In particular, the model tended to favor the dominant class, which means that high overall accuracy alone was not sufficient for evaluating whether the model performed well across both classes.

Therefore, the final model should be interpreted as a preliminary model that learned meaningful information related to the pain-classification task, but still requires further improvement before it can be considered robust or clinically reliable. The selected checkpoint was saved and used for the final test-set prediction stage.

Conclusion:

In this project, a complete machine learning pipeline was developed for binary knee-pain classification using thermal images. The process included clinical tabular-data preprocessing, thermal-image preprocessing, label assignment, patient-level dataset splitting, targeted augmentation, CNN model development, hyperparameter search, final model training, and test-set prediction export.

The final dataset was constructed carefully in order to ensure that each image was matched with the correct patient, visit, knee side, and pain label. Images without a direct clinical match were removed from the modeling dataset, and the final labeled data were split at the patient level in order to reduce the risk of data leakage. Since the original dataset was imbalanced, augmentation was applied only to the minority class within the training set, producing a more balanced augmented training set while preserving the validation and test sets in their original form.

A configurable CNN architecture was then trained to classify the processed thermal images. Hyperparameter search was used to identify a suitable configuration, and the selected model was further trained during the final training stage. The final checkpoint was chosen according to the lowest combined training and validation loss.

Overall, the results show that the model was able to learn information relevant to the pain-classification task. However, the test-set analysis also revealed that the model's predictions were influenced by class imbalance and that its ability to distinguish reliably between pain and no-pain cases was still limited. Therefore, the results should be interpreted as preliminary model-development findings, rather than as evidence of a final clinically reliable model.